Exercise Script: Systematic Data Analysis with Python

Module: Data Engineering

Fundamentals to Advanced Techniques

Winter Semester 2026, P. Kern

Contents

1 Introduction and Organizational Notes **3**
- 1.1 Use of AI . . . 3
- 1.2 The Project: Smart City Sensor Network . . . 4
- 1.3 Learning Objectives . . . 4

2 Module 1: Preparation and Setup **5**
- 2.1 Task 1.1: Create Folder Structure . . . 5
- 2.2 Task 1.2: Create Virtual Environment (venv) . . . 5
- 2.3 Task 1.3: The First Test . . . 5
- 2.4 Review Questions for Module 1 . . . 5

3 Module 2: Variables and Data Types **6**
- 3.1 Task 2.1: Simulate Sensor Data . . . 6
- 3.2 Task 2.2: Check and Print Types . . . 6
- 3.3 Task 2.3: Apply Mathematical Operators . . . 6
- 3.4 Review Questions for Module 2 . . . 6

4 Module 3: Control Structures (If-Statements) **7**
- 4.1 Task 3.1: Check Sensor Status . . . 7
- 4.2 Task 3.2: Multiple Branches (elif) . . . 7
- 4.3 Task 3.3: Logical Operators (and / or) . . . 7
- 4.4 Review Questions for Module 3 . . . 7

5 Module 4: Loops **8**
- 5.1 Task 4.1: The while-Loop (Simulation of a Heating Process) . . . 8
- 5.2 Task 4.2: The for-Loop with range() . . . 8
- 5.3 Task 4.3: Loop Control (break and continue) . . . 8
- 5.4 Review Questions for Module 4 . . . 8

6 Module 5: Data Structures - Lists and Dictionaries **9**
- 6.1 Task 5.1: Create and Manipulate Lists . . . 9
- 6.2 Task 5.2: Iterate Over Lists . . . 9
- 6.3 Task 5.3: Dictionaries . . . 9
- 6.4 Task 5.4: List of Dictionaries . . . 9
- 6.5 Review Questions for Module 5 . . . 9

7 Module 6: Reading CSV Files and Classical Analysis **10**
7.1 Task 6.1: Read File . . . 10
7.2 Task 6.2: Parse Data and Filter Outliers . . . 10
7.3 Task 6.3: Mathematical Evaluation (Standard Python) . . . 10
7.4 Review Questions for Module 6 . . . 11

8 Module 7: Introduction to NumPy **12**
8.1 Task 7.1: Installation and Import . . . 12
8.2 Task 7.2: Create the NumPy Array . . . 12
8.3 Task 7.3: Vectorized Mathematics . . . 12
8.4 Review Questions for Module 7 . . . 12

9 Module 8: Advanced NumPy Analysis (Boolean Masking) **13**
9.1 Task 8.1: Load Data Directly with NumPy . . . 13
9.2 Task 8.2: Apply Boolean Masking . . . 13
9.3 Task 8.3: Retrieve Statistical Metrics Directly . . . 13
9.4 Review Questions for Module 8 . . . 13

10 Module 9: Data Visualization with Matplotlib **14**
10.1 Task 9.1: Install and Import Matplotlib . . . 14
10.2 Task 9.2: A Simple Line Chart . . . 14
10.3 Task 9.3: Subplots (Before / After Comparison) . . . 14
10.4 Review Questions for Module 9 . . . 14

11 Module 10: Advanced Visualization with Seaborn **15**
11.1 Task 10.1: Install Seaborn . . . 15
11.2 Task 10.2: The Boxplot for Outlier Detection . . . 15
11.3 Task 10.3: The Final Report (Combining All Techniques) . . . 15
11.4 Review Questions for Module 10 . . . 15

1 Introduction and Organizational Notes

Welcome to the exercise script for the "Data Engineering" module. In this script, you will practically apply the theoretical knowledge from the lectures. The script is designed as a cohesive project that you can work through over several hours. You will start with simple variables and work your way through loops, control structures, and mathematical operations up to the industry standards of data analysis (NumPy) and visualization (Matplotlib, Seaborn).

1.1 Use of AI

You can use the AI of TH Köln **free of charge** for your questions: `https://chat.kiconnect.nrw/app`

Important: Do **not** just ask for the pure solution, but always for a detailed **explanation**. A mere solution barely supports learning – only a comprehensible explanation promotes understanding.

Example from 3.2:
Question: What are *f-strings* and what can I use them for?

Answer:

f-strings (formatted string literals) were introduced in Python 3.6 and allow embedding Python expressions directly inside string literals. The prefix `f` (or `F`) before the string literal signals that anything inside curly braces `{}` will be evaluated and inserted into the resulting text.

Advantages

- **Readability:** The code is compact and easy to understand.
- **Performance:** f-strings are usually faster than classic %-formatting or `str.format()`.
- **Flexibility:** Any Python expression, format specifier (e.g., `:.2f`), and function call can be used.

Example

```
name   = "Anna"
points = 9.75
print(f"Hello {name}, you achieved {points:.1f} points.")
# -> Hello Anna, you achieved 9.8 points.
```

1.2 The Project: Smart City Sensor Network

You work as a data analyst for a Smart City project. Various sensors are distributed across the city to measure temperature and humidity. Unfortunately, there are disruptions in the network: some sensors deliver implausible values (outliers), while others are temporarily inactive.

Your task is to develop a Python script (e.g., `main.py`) that reads this data, cleans it, evaluates it mathematically, and finally visualizes it professionally.

1.3 Learning Objectives

- Setting up a professional development environment.
- Understanding dynamic typing and the Python Virtual Machine (PVM).
- Mastering control structures (if-statements, loops).
- Working with elementary data structures (lists, dictionaries).
- Algorithmic thinking (filtering, aggregation without external libraries).
- Vectorized computing with NumPy.
- Data visualization with Matplotlib and Seaborn.

2 Module 1: Preparation and Setup

Before we start coding, we need to set up our working environment properly. Python is not just a language, but an entire ecosystem.

2.1 Task 1.1: Create Folder Structure

Create a new folder named `SmartCityProject` on your computer. Open this folder in Visual Studio Code (VS Code). Inside this folder, create a file named `main.py` and a file named `messdaten.csv`. Copy the provided CSV data into this file.

2.2 Task 1.2: Create Virtual Environment (venv)

In the lecture, you learned that a virtual environment (venv) is used to manage packages in isolation from other projects. This prevents version conflicts when Project A requires an older version of a library than Project B.

Tip for setting up a venv:
Open the terminal in VS Code. Enter the following command:
For Windows: `python -m venv .venv`
For Mac/Linux: `python3 -m venv .venv`
Activate the environment afterwards. VS Code often asks directly in the bottom right corner whether this new environment should be used. Click "Yes".
You can also activate the venv via the terminal:
For Windows: `.\.venv\Scripts\activate`
For Mac/Linux: `source .venv/bin/activate`

2.3 Task 1.3: The First Test

Write the following code into your `main.py` to test whether Python works correctly. In the lecture, you learned that Python focuses on extremely readable code (inspired by Monty Python's Flying Circus).

```
1 print("System start: Smart City Analysis")
```

Run the script.
Expected output in the terminal:

```
System start: Smart City Analysis
```

2.4 Review Questions for Module 1

Answer the following questions for yourself (write the answers as comments # in your script):

1. Who invented Python and in which year was it released?
2. What happens in the background when you execute a `.py` file? Explain the terms *Bytecode* and *Python Virtual Machine (PVM)*.
3. Why does it make sense to create a virtual environment (`venv`) in every new project?

3 Module 2: Variables and Data Types

Python uses *dynamic typing*. This means you do not have to declare whether a variable is text (string) or a number (integer). Python detects this automatically at runtime. Four basic data types occur in our CSV file.

3.1 Task 2.1: Simulate Sensor Data

Create four variables in your script that represent a single row from our CSV file. Use the appropriate data types.

- `sensor_id`: The number 1
- `standort`: The text "Nordpark"
- `temperatur`: The number 22.5
- `ist_aktiv`: The boolean value True

3.2 Task 2.2: Check and Print Types

Use the built-in Python function `type()` to determine the data type of each variable. Print the result using `print()`. Use so-called f-strings to embed variables into text.

Expected output in the terminal:

```
Data type of ID: <class 'int'>
Data type of Location: <class 'str'>
Data type of Temperature: <class 'float'>
Data type of Active Status: <class 'bool'>
```

3.3 Task 2.3: Apply Mathematical Operators

We need to convert the temperature to Fahrenheit. The mathematical formula is:

$$Fahrenheit = Temperature \times 1.8 + 32$$

Calculate the value and store it in a new variable `temp_fahrenheit`. Print: "22.5 degrees Celsius corresponds to [X] degrees Fahrenheit."

Expected output in the terminal:

```
22.5 degrees Celsius corresponds to 72.5 degrees Fahrenheit.
```

3.4 Review Questions for Module 2

1. What is the difference between `int` and `float`?
2. What does dynamic typing specifically mean for you as a programmer compared to languages like Java or C++?
3. Which operator is used in Python for exponentiation (e.g., 2 to the power of 3)? (Hint: see lecture slides on operators).

4 Module 3: Control Structures (If-Statements)

In this module, you will learn about conditional branches. An intelligent system must react differently to different sensor values. As discussed in the lecture, Python uses indentation (usually 4 spaces) to define code blocks. There are no curly braces `{}`.

4.1 Task 3.1: Check Sensor Status

Write a simple `if`-statement that checks whether the variable `ist_aktiv` has the value `True`. If so, print "Sensor is sending data". Add an `else` block that prints "Sensor is offline!" if the value is `False`.

Temporarily change the value of the variable `ist_aktiv` to `False` to test both paths.

4.2 Task 3.2: Multiple Branches (elif)

We want to categorize the temperature into different levels to issue warnings. Write an extended control structure (`if`, `elif`, `else`) that checks the following conditions for the variable `temperatur`:

- If the temperature is greater than 50, print "WARNING: Heat outlier detected (Possible defect)!".
- If the temperature is less than 0, print "WARNING: Frost outlier detected!".
- If the temperature is between 0 and 30 (inclusive), print "Temperature in normal range.".
- In all other cases, print "Elevated temperature, but within limits.".

Test your code by setting the `temperatur` variable to 105.0.
Expected output in the terminal (at 105.0):

```
WARNING: Heat outlier detected (Possible defect)!
```

4.3 Task 3.3: Logical Operators (and / or)

Combine conditions. A warning should only be output if the temperature is above 50 **and** the sensor is active. Use the keyword `and`.

Tip on logical operators:
In Python, we use the English words `and`, `or`, `not` to combine boolean values. For example: `if temperatur > 50 and ist_aktiv == True:`

4.4 Review Questions for Module 3

1. Why does missing indentation in Python after an `if`-statement lead to an error (IndentationError)?
2. What does the keyword `elif` do, and when do you use it instead of multiple individual `if`-statements?
3. Which comparison operators do you know? Name at least four (e.g., ==).

5 Module 4: Loops

Often we need to repeat processes. In this module, we learn about `while` and `for` loops.

5.1 Task 4.1: The while-Loop (Simulation of a Heating Process)

Imagine the sensor measures a room that is being heated up. Create a variable `raum_temp = 15.0`. Write a `while` loop that runs as long as `raum_temp` is less than 22.0. In each loop iteration:

- Print the current temperature.
- Increase `raum_temp` by 2.5 degrees.

Expected output in the terminal:

```
Heating... current temp: 15.0
Heating... current temp: 17.5
Heating... current temp: 20.0
Target temperature reached.
```

5.2 Task 4.2: The for-Loop with range()

The `for` loop is ideal for performing a fixed number of repetitions. Use `for i in range(...):` to output the numbers from 1 to 5. Output the square of each respective number (use the exponentiation operator `**`).

Expected output in the terminal:

```
Sensor calibration step 1: 1
Sensor calibration step 2: 4
Sensor calibration step 3: 9
Sensor calibration step 4: 16
Sensor calibration step 5: 25
```

5.3 Task 4.3: Loop Control (break and continue)

Copy your loop from 4.2. Insert an `if`-statement: if the number `i` is exactly 3, this iteration should be skipped (without performing the calculation). Which keyword do you need for this? (`break` or `continue`?). When the number 5 is reached, the loop should be aborted immediately and completely.

> Note: `break` and `continue` were not covered in the lecture. Research and work them out yourself.

5.4 Review Questions for Module 4

1. What is the conceptual difference between a `for` loop and a `while` loop?
2. Explain in your own words the difference between the keywords `break` and `continue`.
3. How do you call the `range` function if you want to start at 0, go up to 10, and count in steps of 2?

6 Module 5: Data Structures - Lists and Dictionaries

So far, we have only worked with individual values. However, a dataset consists of many values. This is where lists and dictionaries come into play.

6.1 Task 5.1: Create and Manipulate Lists

Create a list named `messwerte_liste` containing the following temperatures: 22.5, 23.1, 105.0, 21.8, -5.0, 24.0, 110.5.

- Add a new measurement `23.0` to the end of the list (Hint: `.append()`).
- The value -5.0 is a known measurement error due to a broken cable. Remove it from the list (Hint: `.remove()`).
- Output the length of the list using the `len()` function.

Expected output in the terminal:

```
Updated list: [22.5, 23.1, 105.0, 21.8, 24.0, 110.5, 23.0]
Number of measurements: 7
```

6.2 Task 5.2: Iterate Over Lists

Write a `for` loop that iterates over all values in `messwerte_liste`. Add all values to a variable `gesamtsumme` (which you initialize with 0 before the loop). Afterwards, calculate the average (sum divided by the number of elements) and print it.

6.3 Task 5.3: Dictionaries

While lists are good for sequences of values, dictionaries are perfect for storing structured data with properties. Create a dictionary `sensor_dict` that stores all information of a single sensor (use the structure from the CSV: ID, Location, Temperature, IsActive). Then explicitly access the location and print it out.

Expected output in the terminal:

```
The analyzed sensor is located at: Nordpark
```

6.4 Task 5.4: List of Dictionaries

Combine both concepts. A list can contain various elements, including dictionaries. Create a list that contains two sensor dictionaries. Iterate over the list with a `for` loop and print the following sentence for each sensor: “Sensor [ID] at location [Location] has a temperature of [Temperature]”.

6.5 Review Questions for Module 5

1. Explain the difference between a list (`[ ]`) and a tuple (`( )`). (Hint: mutable vs. immutable).
2. When would you prefer a dictionary over a list?
3. In the lecture, it was mentioned that pure Python lists are slow at math. Which module is introduced later to solve this problem?

7 Module 6: Reading CSV Files and Classical Analysis

Now we read in real data and apply what was presented in the document `002_Beispiel Datenstrukturen - Analyse.pdf`: classical, non-vectorized data analysis using loops.

7.1 Task 6.1: Read File

To read text files (like a `.csv`) in Python, we initially do not need any external libraries. Instead, we use the built-in `open()` function.

It is important that once opened, files are closed again by the operating system to free up memory and avoid file locks. In Python, the so-called `with` block automatically handles this task for us.

Take a look at the following code example:

```
with open('sensor_messdaten_projekt.csv', 'r') as file:
    # Everything indented here happens while the file is open.
    zeilen = file.readlines()

# Here, without indentation, Python has already closed the file automatically.
```

Your Task: Use the code above as a template to read the file `sensor_messdaten_projekt.csv` (or `messdaten.csv`). Subsequently (outside the `with` block), use the `len()` function to determine how many lines are in the list `zeilen` and print this value with an explanatory text using `print()`.

Tip on file processing:
The `with` block ensures that the file is *always* closed automatically after reading, even if an error occurs in the indented code. This is best practice in Python.
By the way, the `'r'` in `open(..., 'r')` stands for *read*. If you wanted to write data to a file, you would have to choose the mode `'w'` (*write*) or `'a'` (*append*).

7.2 Task 6.2: Parse Data and Filter Outliers

Extract exclusively the "Temperature" column from the read lines. Skip the first line (the header). Create two empty lists: `gute_daten` and `ausreisser`. Loop through the extracted temperatures (convert them using `float()`).

- If the value is between 0 and 50, add it to `gute_daten`.
- If the value is above or below that, add it to `ausreisser`.

Expected output in the terminal:

```
Classical analysis started...
Valid data points found: 11
Outliers found: 3
```

7.3 Task 6.3: Mathematical Evaluation (Standard Python)

Now calculate exclusively from the list `gute_daten`:

1. The mean (average).
2. The minimum and the maximum.

Avoid external libraries here; use loops and standard functions as shown in Listing 2 of the lecture.

7.4 Review Questions for Module 6

1. Why does data read from a text file (CSV) have to be explicitly converted with `float()` or `int()` before you can calculate with it?
2. Describe the algorithmic effort: Why is data cleaning using `for` loops inefficient for extremely large datasets (e.g., millions of rows)?

8 Module 7: Introduction to NumPy

Classical analysis with loops works, but it is slow and the code quickly becomes cluttered. As presented in Lecture 002, `NumPy` is the industry standard for scientific computing. It is based on N-dimensional arrays written in C and replaces inefficient for-loops with vectorized operations.

8.1 Task 7.1: Installation and Import

To use NumPy, we need to install it into our virtual environment.

Tip on installing modules:
Open your VS Code terminal (make sure the `venv` is active, recognizable by `(.venv)` before the path). Enter:
`pip install numpy`
Then import it at the top of your `main.py` script with: `import numpy as np`

8.2 Task 7.2: Create the NumPy Array

Take your cleaned list `gute_daten` from Task 6 and convert it into a NumPy array: `daten_np = np.array(gute_daten)`
Check the data type of the new variable using `type()`.
Expected output in the terminal:

```
Type of the NumPy object: <class 'numpy.ndarray'>
```

8.3 Task 7.3: Vectorized Mathematics

Remember Task 2.3 (Celsius to Fahrenheit)? Back then, we converted a single value. With a normal list, we would now need a loop. With NumPy, this works without a loop for all values simultaneously!
Execute the calculation `fahrenheit_np = daten_np * 1.8 + 32`. Print the resulting array.
Expected output in the terminal:

```
Fahrenheit values: [72.5  73.58 71.24 75.2  73.22 74.3  71.78 74.84 71.6  76.1  73.4 ]
```

8.4 Review Questions for Module 7

1. What does the abbreviation NumPy stand for?
2. Which central data object is the heart of NumPy?
3. Explain the term “vectorization” in the context of data processing in Python.

9 Module 8: Advanced NumPy Analysis (Boolean Masking)

In this module, we replicate the data cleaning from Module 6, but this time the modern NumPy way, as described in Listing 3 of the document "002_Beispiel Datenstrukturen - Analyse.pdf".

9.1 Task 8.1: Load Data Directly with NumPy

NumPy can load CSV files directly, saving us error-prone manual parsing. Since our CSV has a text header and mixed data types, we use the function `np.genfromtxt` to specifically read only the temperature column.

Use the following code to create an array with all raw data (including outliers) directly from the file. Make sure that the index at `usecols` matches the temperature column in your CSV (here exemplary column 3):

```
# skip_header=1 skips the header row
# usecols=2 selects the 3rd column (index 2)
rohdaten_np = np.genfromtxt('messdaten.csv', delimiter=',', skip_header=1, usecols
    =2)
```

9.2 Task 8.2: Apply Boolean Masking

Boolean masking allows us to filter arrays based on conditions, completely without `if`-statements or loops.

Create a condition: `maske_gueltig = (rohdaten_np > 0) & (rohdaten_np <= 50)` *(Note: In NumPy, we use the ampersand & instead of the word and.)*

Apply the mask to generate a cleaned array: `bereinigte_daten_np = rohdaten_np[maske_gueltig]`

Print the cleaned array.

9.3 Task 8.3: Retrieve Statistical Metrics Directly

Use NumPy's built-in C-optimized functions to perform the evaluation. Calculate for `bereinigte_daten_np`:

- `np.mean()` for the mean
- `np.median()` for the median
- `np.std()` for the standard deviation

Output the values rounded (Hint: `round(value, 2)`).

Expected output in the terminal:

```
NumPy Evaluation:
Mean: 23.11
Median: 23.05
Standard deviation: 0.89
```

9.4 Review Questions for Module 8

1. Why is `bereinigte_daten_np = rohdaten_np[rohdaten_np <= 50]` more elegant and faster than the method with the empty list and the `for` loop from Module 6?
2. How do the mean and the median differ statistically? Which value is more susceptible to unfiltered outliers?

10 Module 9: Data Visualization with Matplotlib

Numbers in the terminal are good, charts are better. To demonstrate the impact of outliers, we visualize the data using `matplotlib`.

10.1 Task 9.1: Install and Import Matplotlib

Install the module via your terminal.

Tip for installation:
`pip install matplotlib`
Import in Python: `import matplotlib.pyplot as plt`

10.2 Task 9.2: A Simple Line Chart

Create a simple plot for the **raw data** (`rohdaten_np`) to make the outliers visually recognizable. Use `plt.plot(rohdaten_np, marker='o', color='red')`. Add a title with `plt.title()` as well as axis labels with `plt.xlabel()` and `plt.ylabel()`. Display the chart with `plt.show()`.

10.3 Task 9.3: Subplots (Before / After Comparison)

We want to recreate the document `002_Beispiel Datenstrukturen - Analyse.pdf` exactly (Listing 4). Create two charts side by side (subplots).

```
fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(12,5))
```

- Plot 1 (`ax1`): Line chart of the original data (with outliers) in red.
- Plot 2 (`ax2`): Bar chart (`.bar()`) of `bereinigte_daten_np` in blue. (Hint: The x-axis is `range(len(bereinigte`

Set titles and axis labels for both charts. Call `plt.tight_layout()` and `plt.show()` at the end.

Expected Result: A window opens with two graphs side by side. On the left, the red line spikes extremely upwards (above 100) and downwards (below 0). On the right, you see a uniform blue bar chart where all values level off between 20 and 30.

10.4 Review Questions for Module 9

1. What does the command `plt.subplots(1, 2)` do? What do the numbers 1 and 2 mean?
2. Why is it essential to specify axis labels (`xlabel`, `ylabel`) in a chart?
3. What function does `plt.tight_layout()` perform?

11 Module 10: Advanced Visualization with Seaborn

`Matplotlib` is the workhorse, but `Seaborn` builds on top of it and creates highly professional statistical graphics with extremely little code.

11.1 Task 10.1: Install Seaborn

Install Seaborn.

Tip for installation:
`pip install seaborn`
Import: `import seaborn as sns`

11.2 Task 10.2: The Boxplot for Outlier Detection

In statistics, the so-called "boxplot" is the standard instrument for representing the distribution of data and identifying outliers visually and mathematically accurately.

Configure Seaborn to use a clean background design: `sns.set_theme(style="whitegrid")`

Create a new figure. Pass your array with the raw data (`rohdaten_np`) to the boxplot function: `sns.boxplot(y=rohdaten_np, color="cyan")`

Set a title: "Distribution of Sensor Data incl. Outliers" and display the plot using `plt.show()`.

Expected Result: You see a box representing the majority of the data between $\approx$ 22 and 24. Far above (at 105 and 110) and far below (at -5) the actual lines (the whiskers), there are individual points (diamonds). These are the values automatically identified by Seaborn as statistical outliers.

11.3 Task 10.3: The Final Report (Combining All Techniques)

Summarize your entire code from Modules 7 to 10 in a clean, sequential structure:

1. Read in data.
2. Create boolean mask and generate cleaned dataset.
3. Console: Output print statements for mean and median.
4. Visualization: Generate a Matplotlib subplot window showing the Seaborn boxplot of the original data as the left plot and a Seaborn histogram (`sns.histplot`) of the cleaned data as the right plot.

11.4 Review Questions for Module 10

1. Which library is Seaborn based on under the hood? (Read the lecture text on NumPy/visualization).
2. What does a boxplot (also called box-and-whisker plot) fundamentally display?
3. Reflect: You have now covered the path from individual variables and error-prone `for` loops to highly efficient, vectorized arrays and statistical plotting libraries. Why do we start with simple lists and loops in computer science at all if NumPy can do everything much faster?

Congratulations!

You have successfully completed the foundational project and implemented a complete analytical pipeline process in Python.